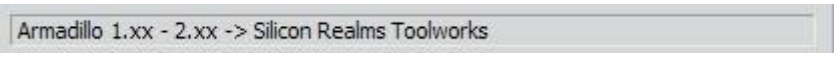
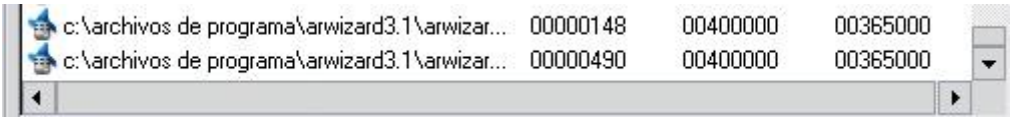
Programa:	ARWizard3.1	
Protección:	Armadillo + CopyMemII y solo graba 90”	
Objetivos:	Desempakar y evitar la limitación	
URL descarga:	www.fu2000.com (creo)	
Dificultad:	No es difícil leyendo las teorías	
Herramientas:	OllyDbg + PEiD + Pupe + Import Reconstructor.	
Cracker:	SyXe'05 [hAcK-c0d3d] [CracksLatinos]	Fecha: 22-08-2005

Hola a todos..!!

Hace no mucho estuve dando un rulo por la red en busca de nuevos programas que nos hagan disfrutar un rato y encontré este pequeñajo, en un principio no iba a escribir el tuto porque lo que aquí se pueda explicar está más que requeteexplicado en las teorías de ***[CracksLatinos]*** pero pensé que porque no dar un ligero repaso a las trampillas que pone el ***Armadillo***, para asegurarnos del protector que es lo abrimos con ***PEiD***, ***RDG***, etc., esto dice ***PEiD***:



Nos dice que se trata de ***Armadillo*** pero no concreta la versión, si lo abrimos con ***RDG*** nos dice ***NADA*** ¿? Bueno igual nos da esto, lo interesante saber aquí es saber que se trata de ***Armadillo*** un viejo conocido por todos nosortos, ahora la cuestión es saber si se crean dos procesos, o sea, si tiene ***CopyMem*** como se dice comúnmente en la jerga, hay mil maneras de detectar esto, la más rápida o almenos para mi es o bien desde el ***Attach*** del ***Olly*** se ven dos procesos o mirar los procesos en ejecución desde ***LordPE***, hagamos esto último:



Está claro que este tiene padre, jeje.. allí se ven los dos procesos, también lo podríamos haber mirado con ***Pupe***, ***ProcDump***, etc., nos quedamos con que se crean dos procesos lo que indica que muy probablemente la presencia de ***CopyMem***, ***Nanomites*** o ambos, hay veces en que el padre no sirve para nada ¿? Si parece increíble pero es cierto ¿queréis pruebas? Mirad el ***Magic Inlay*** que es un juego de ***Oberon Media*** y saldréis de dudas, es más si lo queréis me lo pedís que os lo pasaré encantado. Ahora nuestra misión será determinar la función del padre en el proceso, para esto cargamos ya la víctima en nuestro incondicional amigo ***Ollydbg***, os explicaré un rápido método que medio saqué de un script de un ruso, no sé su nombre sino lo diría, os explico: Armadillo Padre e Hijo como sabemos son el mismo proceso, o sea que el código de ambos es totalmente idéntico, entonces de alguna manera durante la ejecución del Padre debe crear el hijo que luegao depurará, bien cuando le pasa el control a este hijo este se desempaka, construye su ***IAT*** y salta al ***OEP*** pero ¿porqué no crea otro hijo? Si son el mismo proceso!!! La respuesta está en una llamada a la función ***OpenMutexA***…!!! Veamos esta ***API***:

The OpenMutex function returns a handle of an existing named mutex object.

HANDLE ***OpenMutex***(

```
  DWORD dwDesiredAccess,  // access flag
  BOOL bInheritHandle,    // inherit flag
  LPCTSTR lpName          // pointer to mutex-object name
);
```

Return Values

If the function succeeds, the return value is a handle of the mutex object.

Olvidemos de mnomento los parámetros que se le pasan pues no lo queremos estudiar a fondo, nos quedamos con el valor de retorno que será el ***handle*** o manejador del ***objeto Mutex*** en caso de existir o ***0*** si no existe, pues este valor es el que le dice al hijo que no cree otro proceso porque al llamar a esta función será retornado el handle del padre y verá que ya es hijo, pero… y si cargamos al padre en ***Olly*** ponemos un ***BP*** en ***OpenMutexA*** y cambiamos el valor de retorno? Si hacemos esto en la segunda llamada a la api (pues se llama más veces) conseguiremos que el padre se

comporte como hijo..!! esto está muy bien pero al llegar al ***OEP*** si tiene ***CopyMemII*** recibiremos un petardazo porque no hay padre que desencripte el código ;-(sirve en casos en los que como en ***Magic Inlay*** el padre está de farol o en casos de ***Nanomites*** en los que queremos dumpear el proceso, de todas formas no está de más saber esto pues se puede explotar en muchos casos.

NOTA: Digo que cambiemos el retorno en la segunda llamada porque sinó no funciona, esto lo averigüé probando combinaciones y solamente en un ***Armadillo*** me encontré que se debía cambiar en la primera llamada y no en la segunda, imagino que esto tendrá alguna explicación que yo por descontado desconozco. La verdad es que el método no tiene pérdida porque en la llamada que debemos modificar el valor de retorno que estará en ***EAX*** al pulsar ***Ctrl+F9*** y retornar al código de ***Armadillo*** veremos referencias a ***GetProcAddress*** e ***IsDebuggerPresent***.

Una cosa más, si hacemos esto no nos debemos preocupar ya del byte del ***antidebugger*** ya que evitamos la comprobación también ;-)

Y dicho esto retornamos al tema principal, íbamos a mirar la función que cumple el padre, bien cargamos como digo el programa en ***Olly*** y para salir de dudas ponemos un ***BP*** en ***WriteProcessMemory*** y damos ***RUN***, las dos primeras veces que para ni caso pero ojo al ***stack*** en la tercera parada:

```
0012E5D8  005D8406  CALL to WriteProcessMemory from ARWizard.005D8400
0012E5DC  0000004C  hProcess = 0000004C (window)
0012E5E0  00401000  Address = 401000
0012E5E4  00395AF8  Buffer = 00395AF8
0012E5E8  00001000  BytesToWrite = 1000 (4096.)
0012E5EC  0012E6EC  pBytesWritten = 0012E6EC
```

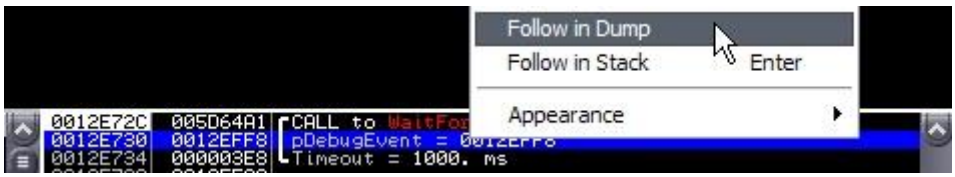
Se dispone a copiar un bloque de ***1000h*** bytes en el proceso hijo a partir de ***:401000*** lo que indica que el ***OEP*** está dentro de este bloque ya que el error se produjo allí. Ahora la pregunta es saber si tiene también ***Nanomites*** que podría ser que los tuviera también luego lo veremos.

Vamos a trabajar, el método para desempaquetar ***Armadillo*** es bastante largo y puede costar de asimilar al principio pero todo queda claro cuando uno hace varios con este sistema, lo que haremos será un breve repaso a lo explicado por ***Ricardo*** en las teorías, para una mayor explicación os remito a esos textos. Vamos:

1.- Esconder el Debugger: Podemos hacerlo o bien con el plugin ***IsDebuggerPresent*** de ***Olly*** que es mucho más cómodo o bien manualmente, esto es poniendo un ***BP*** en la api ***IsDebuggerPresent*** precisamente y cuando ***Olly*** pare ir a la tercera línea, hacer un ***Follow in Dump*** y cambiar el 1 por un 0 que indica ausencia de debugger y así el programa correrá bien:

```
77E52740   64:A1 18000000  MOV EAX,DWORD PTR FS:[18]
77E52746   8B40 30         MOV EAX,DWORD PTR DS:[EAX+30]
77E52749   0FB640 02       MOVZX EAX,BYTE PTR DS:[EAX+2]
77E5274D   C3              RETN
```

2.- Poner un BP en WaitForDebugEvent: esto lo hacemos para que quede el reporte de la api a la vista, la manera que tienen padre e hijo de comunicarse digamos es mediante el uso de la api, el padre queda en un loop entre ***WaitForDebugEvent*** y ***ContinueDebugEvent***, cuando el hijo tiene algún problema suelta un evento que capta el padre mediante ***WaitForDebugEvent***, esta api le pasa un informe con los datos del error que se produjo en el hijo, el primer error como se produce justo en el ***OEP*** en el primer reporte lo veremos también, esta es nuestra baza, entonces ***BP*** en la api y damos ***RUN***, cuando ***Olly*** pare camos al ***stack*** y en el parámetro ***pDebugEvent*** hacemos un ***Follow in Dump*** y ya tenemos el reporte a la vista:



3.- Quitar el Bp de WaitForDebugEvent: Esto es para que quede el reporte a la vista, es importante hacerlo así y rápidamente tendremos el ***OEP***, en la ventana ***Dump*** estará todo.

4.- Poner un BP en la api WriteProcessMemory: Esto es para ver cuando el padre comience a desencriptar bloques del hijo, además cuando pare la primera vez echamos la vista a la ventana ***Dump*** y veremos el ***OEP***:

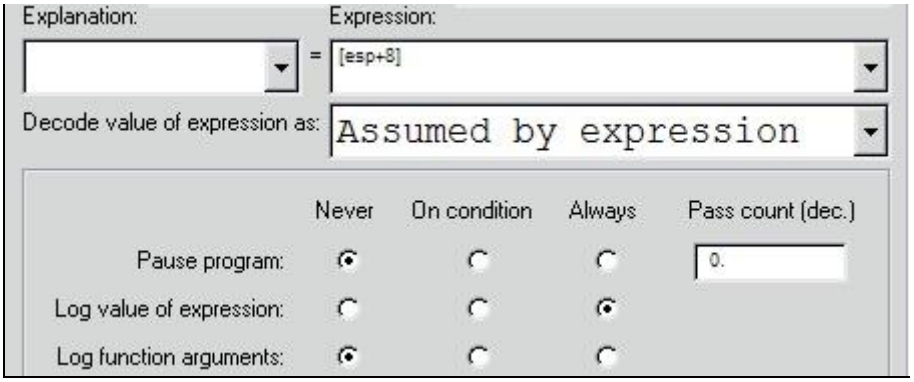
```
Address   Hex dump                                          ASCII
0012EFF8  01 00 00 00 34 0F 00 00 58 06 00 00 01 00 00 80  ☺...4☼..X♠..☺..Ç
0012F008  00 00 00 00 00 00 00 00 00 10 40 00 02 00 00 00  ..........►@.☻...
0012F018  00 00 00 00 00 10 40 00 00 10 40 00 00 00 00 00  .....►@..►@.....
0012F028  01 6A 7A E1 00 00 00 00 18 5D 71 F8 00 00 00 00  ☺jzß....↑]q°....
0012F038  13 00 00 00 38 2D 50 00 64 5D 71 F8 5E CC 4D 80  ‼...8-P.d]q°^╠MÇ
```

Lo que he remarcado es precisamente eso, el ***OEP***..!! como siempre nos lo apuntamos.

OEP: 401000

Ya sabemos además que está compilado en ***Ensamblador***. Sigamos con lo nuestro.

5.- Quitar el Bp de WriteProcessMemory y ponerlo Conditional Log: Lo configuramos de esta manera:



Loggeamos el valor de ***[esp+8]*** que es donde está el valor del bloque a desencriptar y ponemos siempre "***Always***" para que se guarde siempre (todos los bloques)

6.- Nopeo del call destructor de bloques: a veces llamado call encriptador, vamos al ***Call Stack***, veamoslo:

```
Address   Stack     Procedure / arguments                  Called from          Frame
0012E5D8  005D8406  ? kernel32.WriteProcessMemory         ARWizard.005D8400
0012E5DC  0000004C    hProcess = 0000004C (window)
0012E5E0  00401000    Address = 401000
0012E5E4  00395AF8    Buffer = 00395AF8
0012E5E8  00001000    BytesToWrite = 1000 (4096.)
0012E5EC  0012E6EC    pBytesWritten = 0012E6EC
0012E6F8  005D7838  ? ARWizard.005D79AE                    ARWizard.005D7833
0012E6FC  00000000    Arg1 = 00000000
0012E700  00392028    Arg2 = 00392028
```

No se aprecia bien la imagen pero la llamada de arriba es donde estamos ahora, sabemos que está desencriptando un bloque es este momento y que el call desencriptador es el call padre al que estamos, entonces en que a nosotros nos interesa es que he seleccionado, con el botíon derecho elegimos ***Show Call***:

```
005D7833  .   E8 76010000   CALL ARWizard.005D79AE        ARWizard.005D79AE
005D7838  .   83C4 0C       ADD ESP,0C
005D783B  .   25 FF000000   AND EAX,0FF
005D7840  .   85C0          TEST EAX,EAX
005D7842  .v  75 07         JNZ SHORT ARWizard.005D784B
```

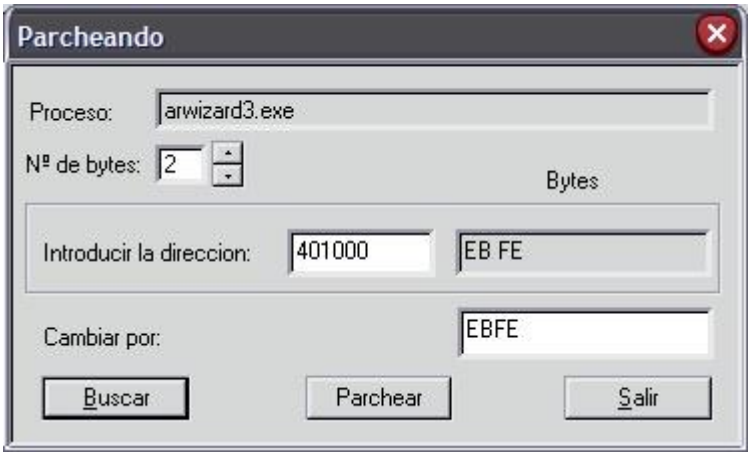
Apareceremos aquí, nos fijamos la dirección donde estamos ***:5D7833***, hacemos Follow (o bien pulsamos Enter es lo mismo) y estamos en el ***PUSH EBP*** de inicio del call ***encriptador/desencriptador***, la diferencia radica en un ***PUSH 0/PUSH 1***, estando sobre el ***PUSH EBP*** del inicio hacemos un ***Ctrl+R*** y que muestre las referencias al call, son dos:

:005D7833 |. E8 76010000 CALL ARWizard.005D79AE
:005D7986 E8 23000000 CALL ARWizard.005D79AE

Vamos a la dirección de la segunda llamada en ***:5D7986*** que es la del call destructor, el otro en ***:5D7833*** es en la que estamos, por eso os decía que nos fijáramos en la dirección del call así ahora lo podemos diferenciar y sabemos cual es el que tenemos que nopear, vamos a ***:5D7986*** y lo nopeamos:

```
005D7986   90   NOP
005D7987   90   NOP
005D7988   90   NOP
005D7989   90   NOP
005D798A   90   NOP
```

7.- Crear un loop infinito en el hijo: Como estamos al inicio de la llamada a ***WriteProcessMemory*** deberemos superarla para que se escriba el primer bloque y con él el ***OEP***, esto lo haremos con ***Ctrl+F9*** una sola vez, después abrimos ***Pupe***, seleccionamos el hijo que es siempre el de arriba de los dos procesos que se muestran y elegimos ***Parchear***, seleccionamos 2 bytes en ***:401000*** y damos a Buscar, aparece ***EB10*** ¿? Vaya ***OEP***, recordad que está compilado en ensamblador y que comienzan con un ***JMP***, vale todo correcto, lugo escribimos ***EBFE*** y pulsamos ***Parchear***, el loop está creado.



8.- Nopeo de WaitForDebugEvent: Ponemos un ***BP*** en la api y cuando pare miramos en el stack la dirección de retorno, vamos hasta allí y hacemos ***New Orign Here*** para evitar que se ejecute la api pues no se debe ejecutar, luego la nopeamos junto con los dos ***PUSH*** que lleva asociados para no descuadrar la pila

```
005D648F   68 E8030000   PUSH 3E8                                  Timeout = 1000. ms
005D6494   8B8D 38FAFFF  MOV ECX,DWORD PTR SS:[EBP-5C8]
005D649A   51            PUSH ECX                                  pDebugEvent
005D649B   FF15 8CF05E0  CALL DWORD PTR DS:[<&KERNEL32.WaitForDel  WaitForDebugEvent
```

Estas son justamente las sentencias a nopear, situamos después el ***EIP*** en ***:5D64C0*** con ***New Origin…*** y cambiamos el salto condicional que hay debajo por un ***JMP*** a una zona libre del exe donde injertaremos una rutina de manera que el padre desencripte todos los bloques del hijo y poderlo dumpear, como siempre forzamos el salto hacia ***:401000*** ojo estamos en el padre ahora..

```
005D64A1     85C0        TEST EAX,EAX
005D64A3   - E9 58ABE2FF JMP ARWizard.00401000
005D64A8     90          NOP
005D64A9     33D2        XOR EDX,EDX
```

Sigue sin verse muy bien pero es un salto hacia ***:401000***,

9.- Realización del injerto: vamos a ***:401000*** y allí injertamos esto:

```
00401000 >  8105 E0EF1200 00100000   ADD DWORD PTR DS:[12EFE0],1000
0040100A    8105 E4EF1200 00100000   ADD DWORD PTR DS:[12EFE4],1000
00401014    8105 10F01200 00100000   ADD DWORD PTR DS:[12F010],1000
0040101E    8105 1CF01200 00100000   ADD DWORD PTR DS:[12F01C],1000
00401028    8105 20F01200 00100000   ADD DWORD PTR DS:[12F020],1000
00401032    8105 4CF01200 00100000   ADD DWORD PTR DS:[12F04C],1000
0040103C    813D E0EF1200 00E04D00   CMP DWORD PTR DS:[12EFE0],dump_1.004DE000
00401046    74 05                    JE SHORT dump_1.0040104D
00401048  - E9 5C541D00              JMP dump_1.005D64A9
0040104D    90                       NOP                    ; Ponemos un BP aquí
0040104E    68 DC020000              PUSH 2DC               ; Handle del hijo (varía en cada ejecución y debe coincidir)
00401053    E8 DBE1A877              CALL kernel32.DebugActiveProcessStop
00401058    90                       NOP                    ; Ponemos un BP aquí
```

Esto teniendo en cuenta que el ***OEP*** aparece en el reporte de ***WaitForDebugEvent*** en las posiciones:

[12EFE0], [12EFE4], [12F010], [12F01C], [12F020], [12F04C]

Por eso el injerto hace referencia a ellas, luego tuvimos en cuenta también que:

Primera seccion (.text): 401000 + DD000 = [401000 - 4DE000] ; Intervalo (tamaño)

Los bytes del injerto son estos:

81 05 E0 EF 12 00 00 10 00 00 81 05 E4 EF 12 00 00 10 00 00 81 05 10 F0 12 00 00 10 00 00 81 05
1C F0 12 00 00 10 00 00 81 05 20 F0 12 00 00 10 00 00 81 05 4C F0 12 00 00 10 00 00 81 3D E0 EF
12 00 00 E0 4D 00 74 05 E9 5C 54 1D 00 90 68 DC 02 00 00 E8 DB E1 A8 77 90

Tenemos puesto el ***BP*** en ***:40104D*** para que al terminar pare allí, antes de comenzar vamos al reporte y ponemos el valor ***:400000*** en las posiciones donde aparece el ***OEP***:

```
Address   Hex dump                                          ASCII
0012EFE0  00 00 40 00 00 00 40 00 00 00 00 00 01 D8 F5 77  ..@...@.....0i$w
0012EFF0  02 00 01 00 F8 EF 12 00 01 00 00 00 34 0F 00 00  ☻.☺.°'♦.☺...4*..
0012F000  58 06 00 00 01 00 00 80 00 00 00 00 00 00 00 00  X♠..☺..Ç.......
0012F010  00 00 40 00 02 00 00 00 00 00 00 00 00 00 40 00  ..@.☻.........@.
```

Esto lo haremos con las 6 posiciones, he visto en otros tutes como sólo modifican 3 pero yo no me fio y prefiero hacerlo así aunque sea un poco más de trabajo pero me aseguro. Ya está casi todo listo, solamente queda guardar el archivo donde irán la dirección de los bloques que se desencriptan, recordad que tenemos un ***BP Conditional Log*** en ***WriteProcessMemory***, picamos en el botón de los puntos suspensivos y con el botón derecho elegimos ***Log to file***, le damos un nombre y listos ya podemos dar ***RUN***, al poco parará en el ***BP*** de ***:40104D*** que indica que ya terminaron de desencriptarse todos los bloques, si miramos el archivo que guardamos veremos allí todos los bloques, si no es así repasar algo que seguramente estará mal algún paso. Cuando acabe si miramos el archivo que hemos guardado veremos todos los bloques, lo mismo en la ***ventana Log*** (***Alt+L***):

```
77E8F13C  Breakpoint at kernel32.WaitForDebugEvent
77E41A94  COND: 00402000
77E41A94  COND: 00403000
77E41A94  COND: 00404000
77E41A94  COND: 00405000
77E41A94  COND: 00406000
77E41A94  COND: 00407000
77E41A94  COND: 00408000
77E41A94  COND: 00409000
          ...   ...
77E41A94  COND: 004D6000
77E41A94  COND: 004D7000
77E41A94  COND: 004D8000
77E41A94  COND: 004D9000
77E41A94  COND: 004DA000
77E41A94  COND: 004DB000
77E41A94  COND: 004DC000
77E41A94  COND: 004DD000
0040104E  Breakpoint at ARWizard.0040104E
```

No los pongo todos, sólo el inicio y el final, se ven todos desde ***:401000*** hasta ***:40DD00*** así que no hubo error, ahora volvemos a dar ***RUN*** para desenganchar el padre del hijo, nos detendremos en ***:401058***, se habrá ejecutado el ***call DebugActivePocessStop*** que separa los procesos cuyo retorno debe ser ***EAX==1*** sino repasar algo que hubo un error también, cuando hayamos hecho esto cerramos ***Olly*** y abrimos uno nuevo donde atachearemos el hijo quedará detenido así:

```
Attached process paused at ntdll.DbgBreakPoint
```

Damos a F9 y luego F12 y queda loopeando en el ***OEP***, jeje.. deshacemos el loop restaurando los bytes originales, ahora es pieza fácil ya que no tiene padre y lo podemos debuggear, vamos a por la ***IAT***, vemos un call a ***GetModuleHandleA***, entramos dentro y vemos la ***JumpTable***:

```
Address   Hex dump        Disassembly                    Comment
004DCFBE  - FF25 44854F00 JMP DWORD PTR DS:[4F8544]      kernel32.GetModuleHandleA
004DCFC4  - FF25 48854F00 JMP DWORD PTR DS:[4F8548]      kernel32.GetOEMCP
004DCFCA  - FF25 4C854F00 JMP DWORD PTR DS:[4F854C]      kernel32.GetPrivateProfileStringA
004DCFD0  - FF25 50854F00 JMP DWORD PTR DS:[4F8550]
004DCFD6  - FF25 54854F00 JMP DWORD PTR DS:[4F8554]      kernel32.GetProcessHeap
```

Vemos punteros correctos y otros que apuntan hacia zonas de ***Armadillo*** que son los que debemos reparar, elegimos uno cualquiera y hacemos un ***Follow in dump->Memory Address*** para caer en la ***iat***, buscamos el inicio hacia arriba hasta llegar a estos punteros:

```
Address   Hex dump                                          ASCII
004F8198  00 10 00 10 F0 2B 00 10 C0 28 00 10 F0 7B 00 10  .►.►-+.►└(.►-{.►
004F81A8  70 22 00 10 90 29 00 10 80 13 00 10 20 20 00 10  p".►É).►Ç‼.►  .►
004F81B8  10 22 00 10 E0 20 00 10 00 70 00 10 C0 72 00 10  ►".►α .►.p.►└r.►
004F81C8  30 6F 00 10 80 26 00 10 80 73 00 10 50 73 00 10  0o.►Ç&.►Çs.►Ps.►
```

Pueden parecernos que no son válidos pero sí que lo son, en concreto son de la librería vorbis.dll que es propia del programa y no común, ahora anotamos el inicio de tabla:

Inicio iat: 4F8198

Sigamos, ahora a por el final, lo haremos de la misma manera, en este caso no es difícil encontrar el final dado que justo después del último puntero están los nombres de las dll's que utiliza:

```
Address   Hex dump                                          ASCII
004F9148  8F 3D 0F 77 E2 19 0F 77 EC 14 0F 77 B8 AB 0F 77  A=*w↑↓*w↓¶*w@½*w
004F9158  00 00 00 00 56 4F 52 42 49 53 2E 44 4C 4C 00 56  ....VORBIS.DLL.V
004F9168  4F 52 42 49 53 45 4E 43 2E 44 4C 4C 00 4F 47 47  ORBISENC.DLL.OGG
004F9178  2E 44 4C 4C 00 46 4D 4F 44 2E 44 4C 4C 00 41 44  .DLL.FMOD.DLL.AD
```

allí vemos lo que os decía, se ve el nombre de las librerías que importa, además están en el mismo orden en que aparecen en la tabla, o sea que los primeros punteros son de ***vorbis.dll*** como ya hemos dicho, los siguientes de ***vorbisenc.dll*** luego los de ***ogg.dll***, etc., y así hasta el último, venga anotamos el final de tabla también:

final iat: 4F9158

Con estos dos datos ya podemos encontrar su largo es una simple resta "***Final - Inicio***" nos da esto:

Longitud iat: FC0

Pero esto no es suficiente porque hay muchos punteros inválidos, el ***Import*** nos dice ***499*** pero hay muchos que son basura así que no sabemos con seguridad cuantos son, pero igual nos da esto ahora, llegó el momento de dumpear el proceso antes de continuar, os diré que si volcamos ahora tal cual está el archivo cuando luego vayamos a reparar la iat con el ***Import Reconstructor*** nos pasará esto ¿?

Este tipo de errores pueden desconcertar ya que no estamos acostumbrados a que nuestras herramientas fallen pero ***Armadillo*** es una excepción en este aspecto ya que utiliza pequeás artimañas para complicarnos el trabajo, por suerte esto tiene muy fácil solución, aquí hay quien coje y pone el ***PE header*** del original y listo pero yo no lo hago así, os explico, si vamos a la sección de Memoria con ***Alt+M*** veremos algo así:

```
00400000 00001000 ARWizard                    Imag R     RWE
00401000 000DD000 ARWizard .text              Imag R     RWE
004DE000 00018000 ARWizard .data              Imag R     RWE
```

No vemos el identificador ***PE header*** como deberíamos ver allí, el motivo es porque ***Armadillo*** en un intento desesperado para no ser dumpeado correctamente nos modifica el ***PE signature*** de la cabecera del archivo, esto lo vemos aquí mejor:

```
0040003B   00        DB 00
0040003C   B81F2200  DD 00221FB8     Offset to PE signature
00400040   BA        DB BA
```

Esta imagen la obtuve yendo a la ventana de memoria haciendo doble clic sobre la sección del header que empieza en ***:400000*** y con el botón derecho elegir ***Special->PE header***, observamos que nuestra signature apunta hacia ***0x221FB8***, esta es la razón de todo, en realidad debería apuntar a la cadena ***PE*** dentro de la propia cabecera:

```
Address   Hex dump                                          ASCII
00400200  50 45 00 00 4C 01 10 00 BA 8F 27 3E 00 00 00 00  PE..L0►.‖A'>....
00400210  00 00 00 00 E0 00 0E 03 0B 01 53 52 00 10 01 00  ....α.♫♥♂0SR.►0.
00400220  00 D0 16 00 00 00 00 00 18 A0 1D 00 00 F0 1C 00  .š_.....↑á#..-L.
```

está en ***:40200*** como trabaja en ***offsets*** debería apuntar hacia ***0x200***, lo único que debemos hacer es ir a la dirección ***:40003C*** y cambiar el valor que haya por un ***200h*** de esta manera:

```
Address   Hex dump                                          ASCII
00400030  00 00 00 00 00 00 00 00 00 00 00 00 00 02 00 00  ..............►B..
00400040  BA 10 00 0E 1F B4 09 CD 21 B8 01 4C CD 21 90 90  ‖►.▲▼┤.=↑☺☺L=↑ÉÉ
00400050  54 68 69 73 20 70 72 6F 67 72 61 6D 20 6D 75 73  This program mus
```

Así está apuntado al lugar correcto ya, si miramos ahora veremos todo en su lugar ;-)

```
003F0000 00002000                          Map  R    R
00400000 00001000 ARWizard          PE header Imag R    RWE
00401000 000DD000 ARWizard .text           Imag R    RWE
004DE000 00018000 ARWizard .data           Imag R    RWE
```

Ahora si que lo reconoce como la cabecera pero antes de seguir tenemos que reparar otro pequeño problemilla:

```
00400200   50 45 00 0 ASCII "PE"        PE signature (PE)
00400204   4C01       DW 014C           Machine = IMAGE_FILE_MACHINE_I386
00400206   1000       DW 0010            NumberOfSections = 10 (16.)
00400208   BA8F273E   DD 3E278FBA        TimeDateStamp = 3E278FBA
```

Nos dice que el número de secciones son ***16*** (***10h***) pero esto no es cierto tampoco y si miramos el archivo desde ***Peditor*** veremos que en realidad tiene sólo ***12***, esto es cosa de ***Armadillo*** también, nada, vamos a la dirección que vemos al lado ***:4000206*** y cambiamos el ***10h*** por ***0Ch*** y listos:

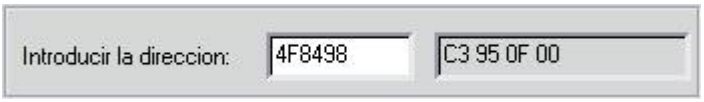
```
Address  Hex dump                                          ASCII
00400206 0C 00 BA 8F 27 3E 00 00 00 00 00 00 00 00 E0 00  ♂.º▒'>........ó.
00400216 0E 03 0B 01 53 52 00 10 01 00 00 D0 16 00 00 00  ♫♥♂☺SR.►☺..$....
00400226 00 00 18 A0 1D 00 00 F0 1C 00 00 70 0F 00 00 00  ..↑á#..-└..p*...
```

Y ahora ya podemos dumpear tranquilamente el proceso, lo podemos hacer con el mismo ***OllyDump*** que va muy bien, lo dejamos como ***dump.exe*** por ejemplo, antes de hacer esto desde la ventana de memoria seleccionamos con el botón derecho la sección ***.idata*** donde irá la iat y le damos acceso total con ***Set Acces->Full Acces***, sino hacemos esto podriamos tener luego problemas con el ***Import***.

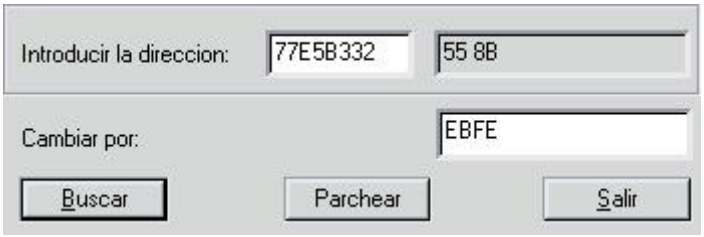
Bien, ahora que tenemos ya el archivo volcado vamos a repararle la tabla, miremos el primer puntero redirigido:

[4F8498]=3B74CF

lo anotamos junto a su valor final, este dato nos servirá ahora para el siguiente paso, lo que haremos será cerrar ***Olly*** y abrir uno nuevo, o bien sin cerrarlo abrir el archivo original protegido con ***Armadillo***, ponemos un ***BP*** en la api ***WaitForDebugEvent***, abrimos ***Pupe*** si lo habíamos cerrado y en el proceso hijo vamos mirando la entrada elegida hasta que detectemos el primer cambio:



Vemos que todavía no ha escrito el valor final, es el momento de hacer un loop infinito en ***GetProcAddress*** que en mi máquina está en ***:77E5B332*** desde el commandbar la puedes obtener en la tuya con ***?GetProcAddress***, en el Pupe parcheamos en el proceso hijo la api haciendo un loop infinito



Pulsamos ***Parchear*** y el loop estará hecho, ahora desde cualquier parte del código desenganchamos el padre del hijo con las sentencias que ya cono cemos:

PUSH Handle_Hijo
CALL DebugActiveProcessStop
NOP <= ***BPX aquí..!!***

El handle del hijo lo podemos mirar desde el mismo Pupe, damos ***RUN*** y al parar en el ***BP*** debe ser ***EAX==1*** como ya sabemos sino repetir el proceso y revisar todo, cerramos ***Olly*** y abrimos uno nuevo vamos a ***File->Attach*** donde elegimos el hijo que estará sólo, para que quede bien attacheado deberemos quitar el loop de la api ***GetProcAddress*** que tenemos restaurando los bytes originales ***558B***, antes de dar a ***F9*** ponemos un ***Bp*** en ***GetProcAddress*** también para parar allí, también vamos al puntero que estamos controlando ***[4F8498]*** y vemos que todavía no tomó su valor

final, jeje.. nos hemos adelantado, le ponemos un ***BPHWrite->Dword*** y damos ***RUN***, ***Olly*** parará al momento, ha saltado el ***BP GetProcAddress***, dulsamos ***Ctrl+F9*** para retornar al código de Armadillo, aparecemos en esta parte del código:

```
Address   Hex dump     Disassembly                      Comment
003B6756    51         PUSH ECX
003B6757    FF15 F8A03C00 CALL DWORD PTR DS:[3CA0F8]    kernel32.GetProcAddress
003B675D  v EB 54      JMP SHORT 003B67B3
003B675F    33C0       XOR EAX,EAX
```

Ahí se ve de donde venimos, si traceamos un poco con ***F8*** llegamos a la zona donde va rellenando la tabla:

```
Address   Hex dump     Disassembly                      Comment
003C125E    8B8D 78FDFFFF MOV ECX,DWORD PTR SS:[EBP-288]
003C1264    8908       MOV DWORD PTR DS:[EAX],ECX       ntdll.RtlDeleteCriticalSection
003C1266    8B85 80FDFFFF MOV EAX,DWORD PTR SS:[EBP-280]
```

En esta pasada termina guardando una api correcta, si traceamos un poco llegamos el final del loop donde va rellenando la tabla que está justo debajo de la zona donde guarda, ahora es cuestión de ir traceando con paciencia en busca del salto mágico que sabemos tiene ***Armadillo***, nuestro siguiente objetivo será buscarlo y modificarlo de manera que se rellene la tabla con direcciones de apis correctas y no con valores hacia la zona de ***Armadillo***. Un buen comienzo suele ser buscar el ***call GetProcAddress*** donde obtiene la dirección de la api, ya que antes de ir a guardar debe estar el salto buscado, el ***call GetProcAddress*** lo vimos antes que es donde paró ***Olly*** y el call padre está en ***:3C120C***, no vemos saltos que eviten pasar por el call así que el mágico debe estar dentro, como hay muchos saltos una manera descrita por ***Ricardo*** en sus tutes de ***Armadillo*** con destrucción de tabla es ir anotando si salta o no salta y si guarda una entrada buena o mala, en el momento que cambie de buena a mala se miran los saltos que cambiaron de comportamiento y daremos con él, en este caso es este:

```
Address   Hex dump     Disassembly                      Comment
003B6678    59         POP ECX
003B6679    85C0       TEST EAX,EAX
003B667B  v 74 0A      JE SHORT 003B6687                Salto Magico..!!
003B667D    83C6 10    ADD ESI,10
003B6680    397E 08    CMP DWORD PTR DS:[ESI+8],EDI
003B6683  ^ 75 DD      JNZ SHORT 003B6662
003B6685  ^ EB B9      JMP SHORT 003B6640
003B6687    8B46 08    MOV EAX,DWORD PTR DS:[ESI+8]
003B668A  ^ EB C1      JMP SHORT 003B664D
```

Para asegurarnos ponemos un ***HE*** en el salto y otro donde guarda en la tabla y vamos mirando, cuando no salta guarda una buena y si salta una mala, es pues el ***SALTO MÁGICO*** y en este caso se debe ***NOPEAR***, pero si lo hacemos ahora ya nos ha metido muchas entradas malas entonces lo que haremos será reiniciar con ***Ctrl+F2***, poner un ***BP*** en ***WaitForDebugEvent*** e ir mirando desde ***Pupe*** los bytes ***740A*** del salto, cuando aparezcan allí los cambiamos por ***EBFE*** que creará un bucle infinito en el salto, desenganchamos el padre del hijo que attacheamos desde otro ***Olly*** nuevo tras cerrar este, quedará así:

```
Address   Hex dump     Disassembly                      Comment
003B667B  - EB FE      JMP SHORT 003B667B
003B667D    83C6 10    ADD ESI,10
003B6680    397E 08    CMP DWORD PTR DS:[ESI+8],EDI
```

Así quedó attacheado desde ***Olly*** detenido en el salto mágico, si miramos ahora la iat:

```
Address   Hex dump                                         ASCII
004F8498  C3 95 0F 00 D1 95 0F 00 E3 95 0F 00 F3 95 0F 00  ├ò*.Ðò*.ãò*.óò*.
004F84A8  01 96 0F 00 0F 96 0F 00 1F 96 0F 00 2F 96 0F 00  ©ü*.*ü*.▼ü*./ü*.
004F84B8  47 96 0F 00 55 96 0F 00 6D 96 0F 00 81 96 0F 00  Gü*.Uü*.mü*.üü*.
```

Como vemos está intacta, nopeamos el mágico y damos ***RUN***, podemos poner un ***BPM*** en la última entrada de la tabla pero nos da igual ahora necesitamos una tabla nueva independientemente del proceso ya que cuando está lista nos iremos al Import que nos reparará el dumpeado, le damos los datos correctos:

```
IAT Infos needed
OEP  00001000   IAT AutoSearch
RVA  000F8198   Size  00000FC0
```

Y cuando damos a ***Get Imports*** nos da ***480*** entradas malas! Como es podible? Pero si las miramos con calma veremos que son entradas basura, bouuuuuuuf! nada entonces hacemos ***Cut Thunks*** y reparamos com ***Fix Dump***, nos creará ***dump_.exe*** al que le añadió una nueva sección de ***3000h*** bytes a partir de ***:765000*** que si ejecutamos veremos que funciona a la perfección, lo que indica que no tiene ***Nanomites***;-)

Eliminaremos ahora la ***limitación de 60"*** en la grabación, cuando ejecutamos el programa vemos esta nag:

Es como un baner, tenemos la opción de registro que no podremos ya utilizar porque es ***Armacces.dll*** la encargada de la validación del serial, entonces atacaremos por la directa, jeje.. buscaremos la zona de creación de la ***nag*** y buscaremos un modo de evitarla, luego miramos qué condiciones se deben dar para que esto pase, normalmente suele ser un call que debe retornar cierto valor o bien un ***flag*** que debe contener cierto valor, si nos encontramos en alguno de estos casos tendremos al programa bien pillado, vamos a por la ***nag***…

Cargamos el programa en ***Olly*** y vamos traceando con ***F8*** hasta que aparezca en pantalla, cuando lo haga comenzamos a buscar algún salto que evite pasar por el ***call culpable***, si no lo hay pues nada ***BPX*** en el call y a reiniciar se ha dicho con ***Ctrl+F2***, daremos ***F9*** para llegar hasta el call nuevamente en el que ahora entramos con ***F7*** y atracear con ***F8*** así hasta dar en el clavo, estos son los resultados:

```
0040135D   E8 02C00700        CALL dump_1.0047D364          ; Call que muestra la nag
0047D395   FF57 2C            CALL DWORD PTR DS:[EDI+2C]    ; Subcall  donde se muestra la nag
00401A12   FF92 D8000000      CALL DWORD PTR DS:[EDX+D8]    ; Subcall que crea la Nag por partes en un loop..!!
```

El call a a evitar es el que hay en ***:401A12*** hasta él no debemos llegar que es precisamente el call que buscábamos desde el principio ya que esta es la zona de creación de la ventana, para colmo justo encima tiene un salto:

```
Address   Hex dump        Disassembly                        Comment
004019E8  80BA 9E040000 00 CMP BYTE PTR DS:[EDX+49E],0       Bonito Flag
004019EF  75 54           JNZ SHORT pr_.00401A45             Saltamos por encima..!!
004019F1  8B0D 24104F00   MOV ECX,DWORD PTR DS:[4F1024]      pr_.004F1694
004019F7  8B09            MOV ECX,DWORD PTR DS:[ECX]
004019F9  B2 01           MOV DL,1
004019FB  A1 849E4E00     MOV EAX,DWORD PTR DS:[4E9E84]
00401A00  E8 67DC0200     CALL pr_.0042F66C
00401A05  8BD8            MOV EBX,EAX
00401A07  A1 1C104F00     MOV EAX,DWORD PTR DS:[4F101C]
00401A0C  8918            MOV DWORD PTR DS:[EAX],EBX
00401A0E  8BC3            MOV EAX,EBX
00401A10  8B10            MOV EDX,DWORD PTR DS:[EAX]
00401A12  FF92 D8000000   CALL DWORD PTR DS:[EDX+D8]         Crea la nag y la muestra..!!
00401A18  8B0D 1C104F00   MOV ECX,DWORD PTR DS:[4F101C]      pr_._Form6
00401A1E  8B19            MOV EBX,DWORD PTR DS:[ECX]
```

el salto lo controla ***[EDX+49E]*** que hace referencia a ***[FA259A]*** que está a ***0*** como se ve en la imagen siguiente:

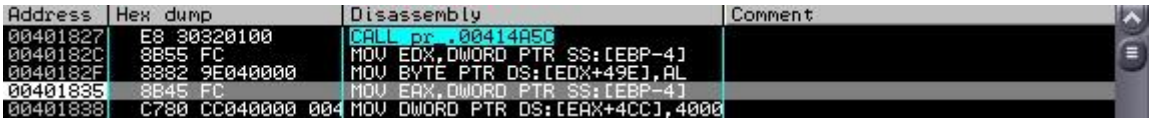
```
Address   Hex dump                                          ASCII
00FA2596  00 00 02 00 00 00 18 50 FA 00 02 00 00 00 C0 71  ■.☺...↑P·.☺...└q
00FA25A6  FA 00 1C 72 FA 00 40 72 FA 00 64 72 FA 00 44 AC  ·.∟r·.@r·.dr·.D¼
00FA25B6  00 00 02 00 00 00 C8 56 FA 00 7F 7F 00 00 00 40  ..☺...╚V·.⌂⌂...@
```

el byte seleccionado en gris es el que lee el programa al llegar justo antes del call que muestra la ventana, si se nos ocurre por ejemplo cambiarle el valor por un 1 además de no ver la nagscreen no pone el botón ***Buy Now***!! Y en el ***About*** nos reconoce como ***Usuario Registrado***, de momento pongamos un ***BreakPoint Hardware on Acces->Byte*** y reiniciemos, la primera vez que para ni caso porque estamos en una instrucción tipo ***REP MOVSD***, pero ojo a la segunda porque esta sí que es más interesante ya:

```
Address   Hex dump        Disassembly                        Comment
0040181D  8B55 FC         MOV EDX,DWORD PTR SS:[EBP-4]
00401820  C682 9E040000 00 MOV BYTE PTR DS:[EDX+49E],0
00401827  E8 30320100     CALL pr_.00414A5C
```

Nos está poniendo el ***flag*** a ***0***, justo antes viene una comprobación y un salto que si tocamos:

jeje no le gustó el cambio, no pasa nada igual seguiremos a lo nuestro, reiniciamos y al llegar a la zona no tocamos nada esta vez damos ***RUN*** y seguimos son el ***BP*** puesto, pero que vemos.. un poco más abajo hay esto:



```
Address   Hex dump        Disassembly                      Comment
00401827  E8 30320100     CALL pr_.00414A5C
0040182C  8B55 FC         MOV EDX,DWORD PTR SS:[EBP-4]
0040182F  8882 9E040000   MOV BYTE PTR DS:[EDX+49E],AL
00401835  8B45 FC         MOV EAX,DWORD PTR SS:[EBP-4]
00401838  C780 CC040000 004 MOV DWORD PTR DS:[EAX+4CC],4000
```

Ahora está tomando el calor de ***AL*** y éste lo toma del ***call 414A5C*** que hay justo antes, esta sí es una zona interesante y lo mejor de todo es que podemos manipular el call para que retorne ***AL==1*** pues de aquí en adelante ya no para más a la que damos ***RUN*** sale la nag en pantalla, o sea que es ahí donde debemos actuar, reiniciamos y ponemos un ***BP*** en ***:401827*** para ver qué hace en el call, primero hace una de sus ya famosas a ***GetEnvironment*** donde busca la variable "***BONW***" el retorno es lógicamente ***EAX==0*** porque no existe tal variable, veamos esto:

```
:00414AF6   E8 A17B0C00      CALL pr_.004DC69C                ; Aqui se decide todo..!!
:00414AFB   50               PUSH EAX                         ; PUSH decisivo..!!
:00414AFC   FF4B 1C          DEC DWORD PTR DS:[EBX+1C]
:00414AFF   8D45 F4          LEA EAX,DWORD PTR SS:[EBP-C]
:00414B02   BA 02000000      MOV EDX,2
:00414B07   E8 AC7A0C00      CALL pr_.004DC5B8                ; No verifica nada aqui..!!
:00414B0C   59               POP ECX                          ; Recupera el PUSH EAX anterior..!!
:00414B0D   84C9             TEST CL,CL                       ; es CL==0?
:00414B0F   74 1F            JE SHORT pr_.00414B30            ; Si es así salta..
:00414B11   B0 01            MOV AL,1                         ; Flag de éxito activado..!!
:00414B13   BA 02000000      MOV EDX,2
:00414B18   50               PUSH EAX
:00414B19   8D45 FC          LEA EAX,DWORD PTR SS:[EBP-4]
:00414B1C   FF4B 1C          DEC DWORD PTR DS:[EBX+1C]
:00414B1F   E8 947A0C00      CALL pr_.004DC5B8
:00414B24   58               POP EAX
:00414B25   8B13             MOV EDX,DWORD PTR DS:[EBX]
:00414B27   64:8915 00000000 MOV DWORD PTR FS:[0],EDX
:00414B2E   EB 1D            JMP SHORT pr_.00414B4D
:00414B30   33C0             XOR EAX,EAX                      ; Flag de error activado..!!
:00414B32   BA 02000000      MOV EDX,2
```

Cuidado con estas zonas que nos pueden confundir un poco al principio, parece que va a validar algo en ***:414B07*** pero no es así el pescado está vendido desde el ***PUSH EAX*** anterior, luego el call anterior es la clave de todo, si entramos en él veremos que está comparando la cadena "***TXRE***" con "h." que se ven en la ventana ***Dump***, aquí lo lógico sería entender como se crea la segunda de ellas, la de "***h.***" ya que la primera se ve claramente que la ercoge en la propia rutina padre que estamos traceando, podríamos poner un BP en la zona a ver donde escribe este valor "***h.***" en la ventana ***Dump*** justo en la posición ***[FAA670]*** cuyo contenido es ***68FE*** que compara con ***54585245*** que son los valores ASCII de "***TXRE***" y los otros de la cadena "***h.***" en realidad no es un punto el que acompaña a la "h" sino un carácter raro pero esto no importa. Si ponemos el ***BPM*** vemos que recoge el valor en un ***REP MOVSD*** y seguir esta línea puede ser costosa además está trabajando constantemente en una zona de memoria teporal que crea en cada ejecución, entonces optaremos por parchear la rutina y nos registre.

Como la zona de la comparación es muy solicitada a lo largo del programa y no nos queremos meter en más líos lo que haremos será parchear el call que realiza nuestra llamada en la que conpara el ***TXRE*** con el h. esta sabemos que está en ***:414AF6***, será cuestión de ir allí y modificar lo que sea necesario, no sin antes verificar como queda la pila (registro ***ESP***) antes y después de la llamada para ver si deberemos reajustarla o no, esto se hace como digo tomando el valor de ***ESP*** justo antes del call y luego justo después y comparando, en este caso vale igual por lo que

ya podemos decir que el call no lo modifica y se puede nopear o cambiar por lo que sea tranquilamente, cambiaremos entonces lo siguiete:

```
Address   Hex dump     Disassembly                          Comment
00414AF3   8D45 FC     LEA EAX,DWORD PTR SS:[EBP-4]
00414AF6   E8 A17B0C00 CALL pr_.004DC69C
00414AFB   50          PUSH EAX
00414AFC   FF4B 1C     DEC DWORD PTR DS:[EBX+1C]
```

POR:

```
Address   Hex dump     Disassembly                          Comment
00414AF3   8D45 FC     LEA EAX,DWORD PTR SS:[EBP-4]
00414AF6   B8 01000000 MOV EAX,1
00414AFB   50          PUSH EAX
00414AFC   FF4B 1C     DEC DWORD PTR DS:[EBX+1C]
```

no se ve muy bien pero es un ***"MOV EAX, 1"*** lo que hay modificado en ***ROJO*** que es nuestro cambio, este es el valor que el programa esperaba y no recibía pues era ***EAX==0***, ahora sí lo recibirá y se comportará del modo que nos conviene más a nosotros ya que pensará que todo está en orden:

```
:00401827   E8 30320100     CALL pr_.00414A5C                 ; Hemos parcheado aquí dentro..!!
:0040182C   8B55 FC         MOV EDX,DWORD PTR SS:[EBP-4]
:0040182F   8882 9E040000   MOV BYTE PTR DS:[EDX+49E],AL      ; Le pasará un 1 al flag como queremos.!!
```

Entonces saldremos de la zona con el ***flag*** inicializado a ***1*** que ya sabemos que nos viene perfecto puesto que ya antes lo comprobamos, si damos ***RUN*** ahora ***Olly*** sí para cosa que antes nos hacía, estamos aquí:

```
Address   Hex dump       Disassembly                          Comment
00407297   E8 10300800   CALL pr_.0048A2AC
0040729C   C683 9E040000 01 MOV BYTE PTR DS:[EBX+49E],1
004072A3   8B4D D4       MOV ECX,DWORD PTR SS:[EBP-2C]
004072A6   64:890D 00000000 MOV DWORD PTR FS:[0],ECX
```

jeje, ahora chafa el 1 que tenemos por otro 1, a esta zona antes no llegábamos lo que indica claramente que se está ejecutando en ***Modo Registrado***, si doy ***RUN*** no para más, carga el programa sin mostrar la nag ni nada, el botón ***Buy Now!!*** ha desaparecido y en la ventana About vemos esto:

Está claro verdad? jeje.. el programa ha cedido a nuestros encantos y nos ha registrado, pues nada hacemos efectivo el cambio y el programa queda bien crackeadito, un ***Armadillo*** menos en nuestro camino ;-))

Y damos por finalizado el tuto ya que los objetivos han sido cubiertos, si hay alguien que tiene alguan duda, o una pregunta o lo que quiera sabe que me puede encontrar en cualquiera de estas direcciones:

syxe00@yahoo.es
syxe00@hotmail.com

No hace falta que diga que todos los mensajes recibidos los contestare, terminamos por hoy pero bienpronto volveremos al ataque con nuevos programas eso seguro, hasta otra…Chao..!!

Dedico el tuta al estupendo cracker ***+NCR/CRC! [ReVerSeR]*** y como no a ***Ricardo*** y en general a toda la lista de ***[CracksLatinos]*** de la que aprendo cada día y también formo parte, aupa compañeros..!!